

BPC-PRP Project Manhattan

1st Matúš Pitoňák

Faculty of Information Technology
Brno University of Technology
Brno, Czech Republic
xpitonm00@vutbr.cz

2nd Samuel Poul

Faculty of Information Technology
Brno University of Technology
Brno, Czech Republic
xpoulsa00@vutbr.cz

Abstract—This paper presents an autonomous maze-navigation system for a differential-drive mobile robot equipped with a 360° LiDAR, wheel encoders, and an RGB camera for ArUco marker detection. The SLAM pipeline combines an occupancy grid with Gauss-Newton scan matching and Monte Carlo localization (MCL). A PCA-guided corridor follower with junction detection handles maze traversal, while a Dynamic Window Pure Pursuit (DWPP) navigator was developed for general navigation. The robot escaped the 8×8 cell evaluation maze with a best time of 30 s including ArUco treasure collection.

Index Terms—Robotics, BPC-PRP, ROS2, OpenCV, LiDAR, SLAM, DWPP

I. INTRODUCTION

Autonomous mobile robotics requires tight integration of perception, state estimation, and planning. The *BPC-PRP* course at Brno University of Technology addresses this progressively: students first implement sensor-based *line following* before tackling autonomous *maze navigation*.

The maze is an 8×8 grid of 40 cm×40 cm cells. The robot starts at the center and must find the exit, collecting ArUco-marked treasures (worth a 30 s time bonus) along the way. No external positioning is available; the robot relies entirely on a 360° LiDAR and wheel encoders.

This paper describes the SLAM pipeline (occupancy grid, Gauss-Newton scan matching, MCL), two navigation strategies (corridor-based reactive driving and DWPP), and key real-world challenges encountered during deployment.

II. SYSTEM ARCHITECTURE

The software is built on ROS2 and organized as an event-driven modular system. A central App owns a typed EventBus for zero-coupling inter-component messaging and a ConfigManager that hot-reloads TOML configuration files at runtime.

III. KINEMATICS AND ODOMETRY

A differential drive kinematic model converts encoder tick counts to linear wheel displacements and integrates them into a 2D pose (x, y, θ) . Velocity commands (v, ω) are inverted to individual wheel setpoints regulated by PID controllers via PWM. An exponential low-pass filter ($\tau=0.1$ s) smooths the reported twist.

IV. SLAM AND LOCALIZATION

A. Occupancy Grid Mapping

The environment is a 200×200 cell grid at 5 cm resolution. Each LiDAR scan is integrated via Bresenham ray-casting: traversed cells are marked *free*, hit cells *occupied*, with probabilities clamped to prevent false certainty.

B. Gauss-Newton Scan Matching

Given pose $\mathbf{x}=(x, y, \theta)^\top$, each scan point is transformed to the world frame and the occupancy value retrieved by bilinear interpolation. Gauss-Newton iterations minimize $\sum_i (1-m(\mathbf{p}_i))^2$, accumulating the Hessian $\mathbf{H}=\mathbf{J}^\top \mathbf{J}$ and applying the correction $\Delta \mathbf{x}=\mathbf{H}^{-1} \mathbf{J}^\top \mathbf{r}$. Match confidence is derived from the eigenvalue structure of $\mathbf{H}$.

C. Monte Carlo Localization

Symmetric corridors create pose ambiguity. MCL [2] maintains a pool of hypotheses, each refined by the scan matcher, governed by a four-state machine (*Initializing* → *Stable* → *Degraded* → *Lost*). On degradation, new hypotheses are spawned by perturbing the best pose within ± 0.75 m XY and $\pm \pi$ rotation; a 10 s lost timeout triggers a full map reset. Fig. 1 shows the hypothesis cloud during an active localization event.

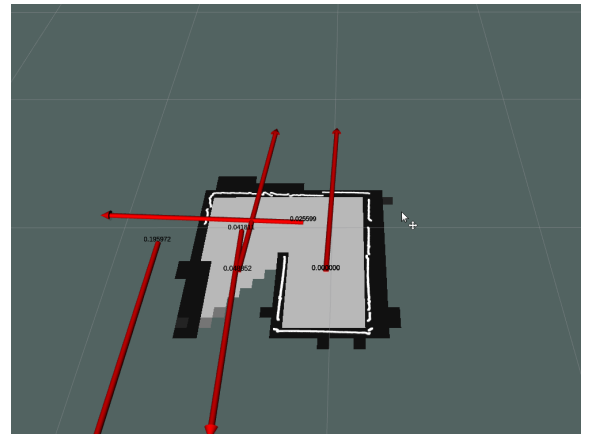


Fig. 1. MCL visualization in RViz2. The displayed text is computed from the likelihood field and is shown in meters.

D. Likelihood Field (Attempted)

A likelihood field—an exponential distance transform of the occupied cells—was developed as an alternative hypothesis scoring function. Fig. 2 illustrates the field over a partially explored map.

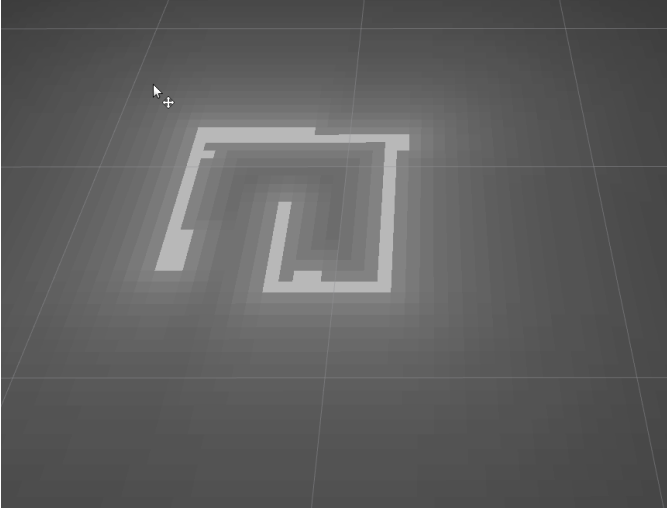


Fig. 2. Likelihood field derived from the occupancy grid using exponential distance scaling. Bright regions correspond to locations close to occupied cells (high likelihood), while darker regions indicate increasing distance from obstacles (low likelihood). The field values are normalized to the range $[0, 1]$.

The method improved robustness in slow, controlled experiments but was sensitive to rapid rotations and the 5 Hz LiDAR update rate, occasionally producing misleading scores during fast maneuvers. It was excluded from the final real-world evaluation, as the Gauss-Newton matcher alone provided sufficient stability at operational speeds. In recovery cases, eigenvalue-based confidence degraded quickly and often led to loss of tracking and rescan triggers, whereas the likelihood-based score typically recovered after a brief loss and returned to a stable pose.

V. NAVIGATION

A. Corridor-Based Maze Navigation

At each 10 ms tick, MazeEngine casts 9 rays per side ($\pm 20^\circ$). PCA fits wall segments; two walls are accepted when their directions are similar. A PD controller drives the heading error relative to the wall bisector; forward speed is modulated by $v = v_{\max} \cdot \text{clamp}(d_{\text{front}}/d_{\text{corridor}}, 0, 1)^2$. Junction detection triggers a *Turning* state executing a $90^\circ/180^\circ$ PI-controlled rotation. Fig. 3 shows the debug view from the final evaluation run.

B. Dynamic Window Pure Pursuit (DWPP)

A DWPP navigator [1] samples (v, ω) candidates within the reachable dynamic window, forward-simulates each trajectory, and scores them by lookahead distance and heading alignment. Parameter tuning was still in progress and was abandoned before completion due to unfinished fine-tuning of variables. The approach was evaluated in a hybrid Unity+ROS2 simulation

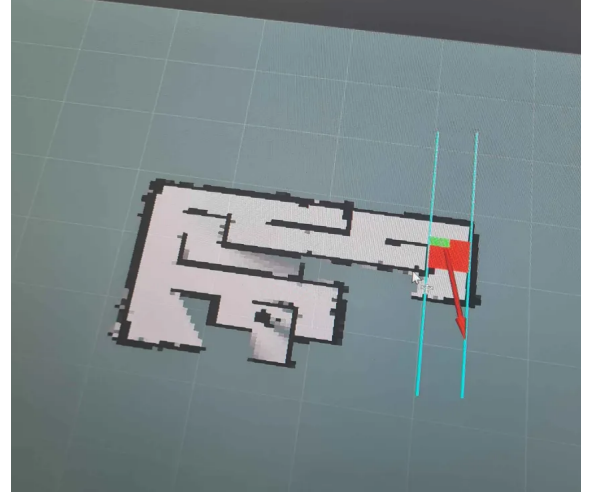


Fig. 3. Corridor navigation debug view. Green lines: PCA-fitted walls. Yellow rays: junction detection casts.

bridge designed for safe parameter exploration. Although not carried through to final evaluation, this method has a notable potential advantage: if fully realized, it could enable the robot to avoid costly 180-degree in-place rotations by allowing more natural reverse motion, improving efficiency in constrained corridor navigation.

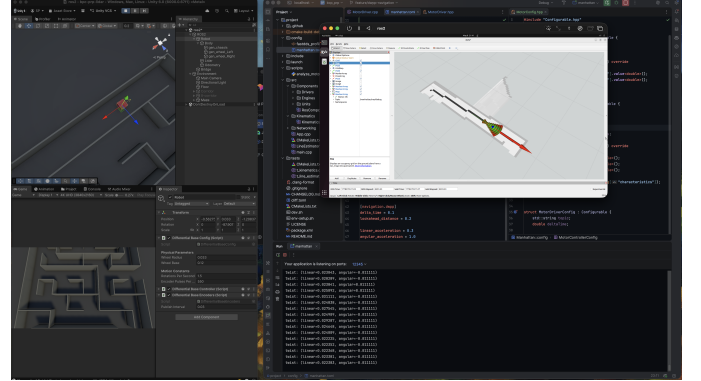


Fig. 4. DWPP tuning in the Unity simulation environment. The RViz2 overlay shows a 10×10 candidate velocity grid. Color (red $\rightarrow$ green) encodes score; the virtual robot ran live ROS2 software against Unity physics.

C. Map Corruption

Fast sudden changes of angular velocity caused little fuzziness and lost of robots orientation in environment. The degenerated map structure caused Gauss-Newton confidence to decrease which led to map corruption as conflicting scans were integrated at incorrect poses (Fig. 5).

After refining the MCL confidence thresholds and hypothesis management, the open-space scenario became paradoxically more stable than narrow corridors: the larger field of view provided more LiDAR return points per scan, yielding a richer Hessian and higher scan-matcher confidence (Fig. 6).

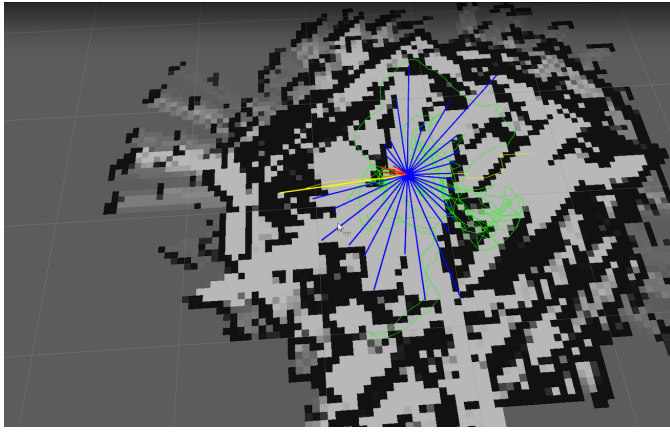


Fig. 5. Corrupted occupancy grid resulting from localization failure in an open space. Multiple conflicting scan integrations create ghost obstacles and erase real walls, while the system attempts further recovery—making the map progressively worse.

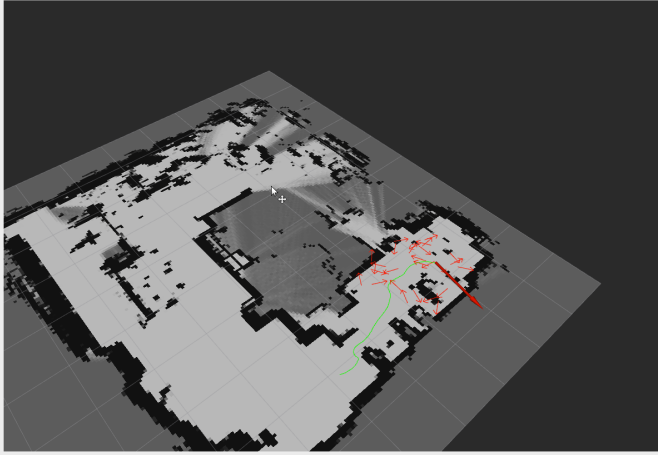


Fig. 6. Robot navigating an open space after MCL refinement. The occupancy grid remains consistent despite the initially problematic sparse, symmetric environment.

VI. RESULTS

The robot successfully escaped the 8×8 maze in all final runs, achieving a best time of ≈ 30 s including ArUco treasure collection and ≈ 50 s in standard runs. Localization was stable at $v \leq 0.4$ m/s $\wedge \omega \leq 2$ rad/s; higher speeds exposed the 5 Hz LiDAR limitation, occasionally triggering the Lost recovery.

Key findings: (1) LiDAR data itself is not inherently limiting, but requires careful transformation of scan points using odometry (encoder-based, with potential future fusion with IMU, which was previously abandoned due to tuning complexity); (2) the more critical issue is scan inconsistency at higher linear and especially angular speeds, where a single LiDAR sweep becomes geometrically distorted, degrading state estimation and control performance. This may be partially fixed by correcting scans using estimated motion/acceleration derived from odometry; (3) ROS2 latency introduces additional challenges for real-time control loops.

VII. CONCLUSION

Project Manhattan is a complete ROS2 autonomous maze-navigation system featuring Gauss-Newton scan matching, MCL, PCA-guided corridor navigation, and DWPP. The robot reliably solved the maze task under real-world conditions.

ACKNOWLEDGMENT

The authors thank the BPC-PRP course staff at FIT BUT for providing the robot hardware, maze environment, and technical guidance.

REFERENCES

- [1] Fumiya Ohnishi and Masaki Takahashi. “Title from the paper”. In: *arXiv preprint arXiv:2601.15006* (2026). Accessed: 2026-05-07. URL: <https://arxiv.org/abs/2601.15006>.
- [2] Wikipedia contributors. *Monte Carlo localization* — *Wikipedia, The Free Encyclopedia*. Accessed: 2026-04-22. 2026. URL: https://en.wikipedia.org/wiki/Monte_Carlo_localization.